

What is Testing?

- A Difficult Question to Answer
- Building confidence
- Prevent defects
- Detect defects
- Ensure all client requirements are met
- Risk Management
- Improving the quality

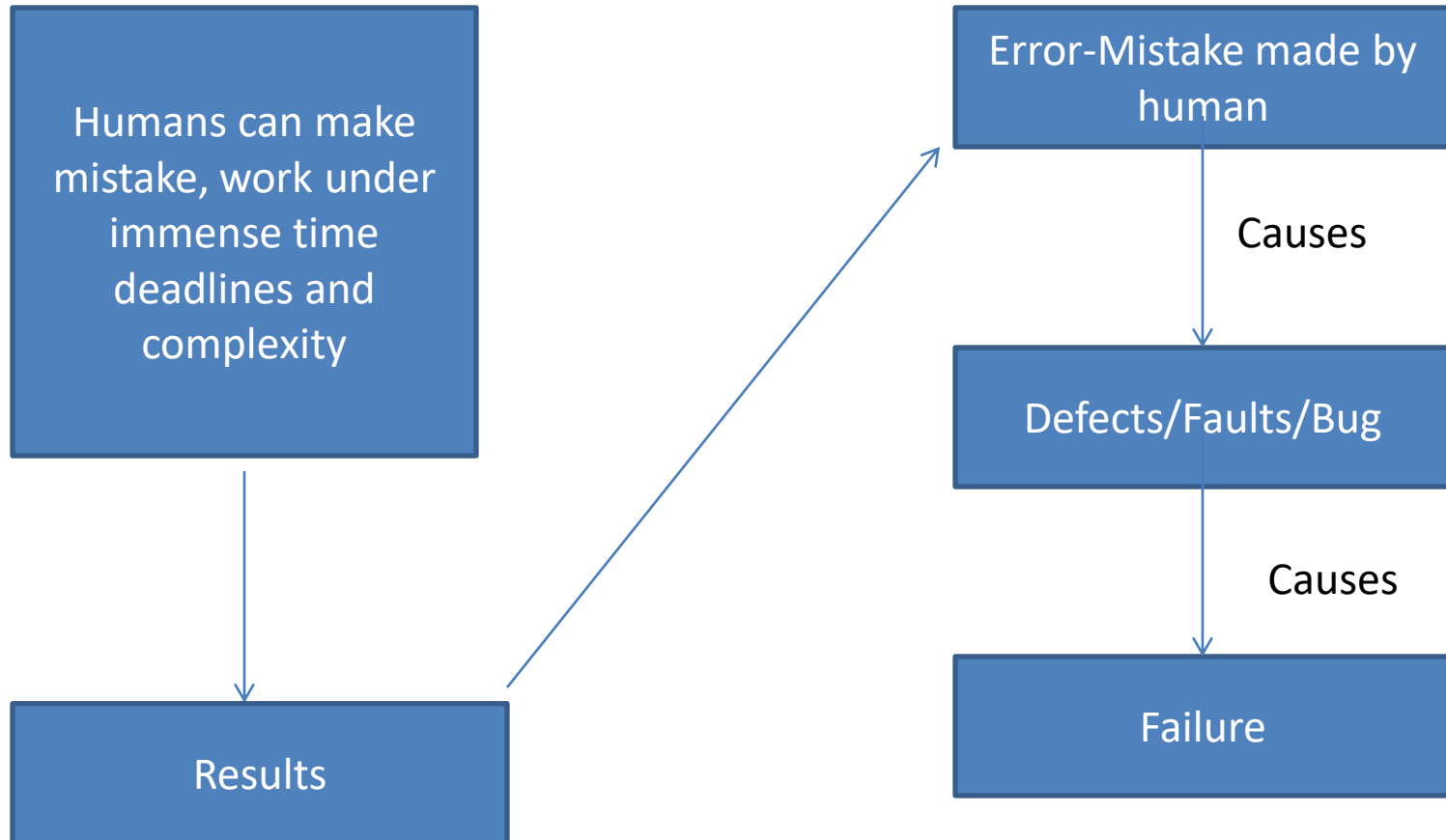
Why Testing?

- 2 viewpoints on testing – **positive** and **negative/attack**
- Positive- Software meets requirements and requirements meet user needs
- Negative- Force specific failures and try to break the system.

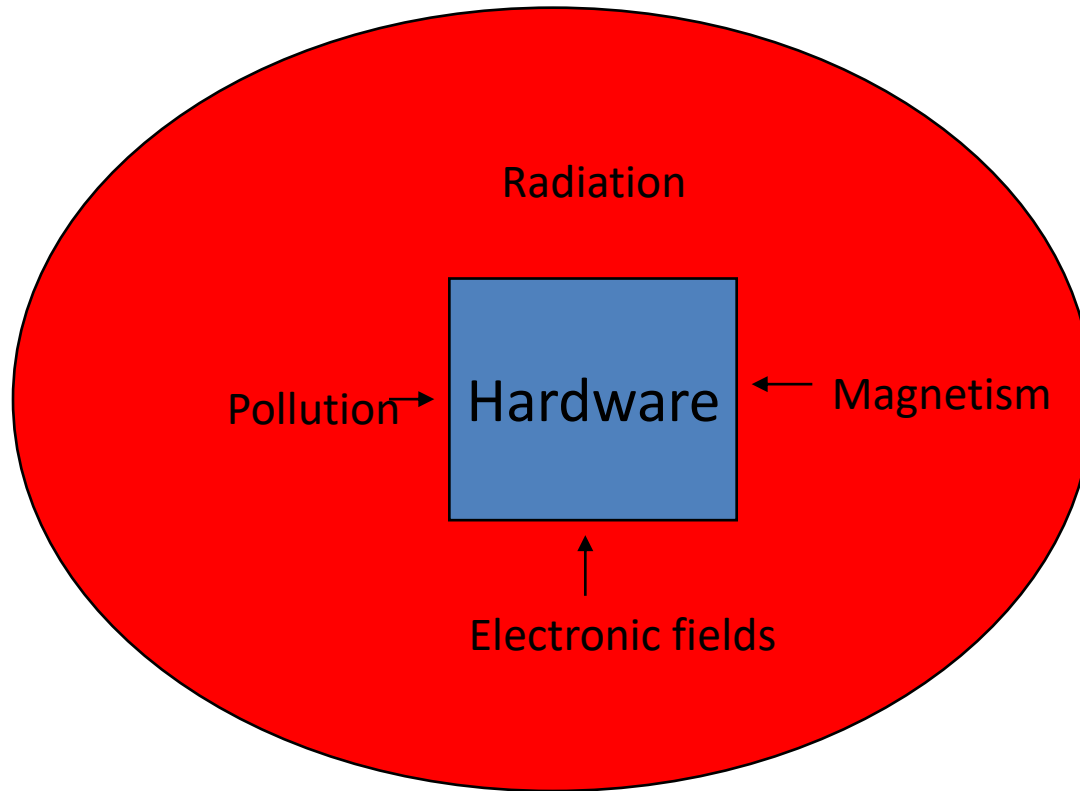
From Software point of view

- Software that is found to be scrap
 - **Create following issues at organization levels-**
 - Loss of money
 - Loss of time
 - Loss of business reputation
 - Injury
 - Death

Causes of defects



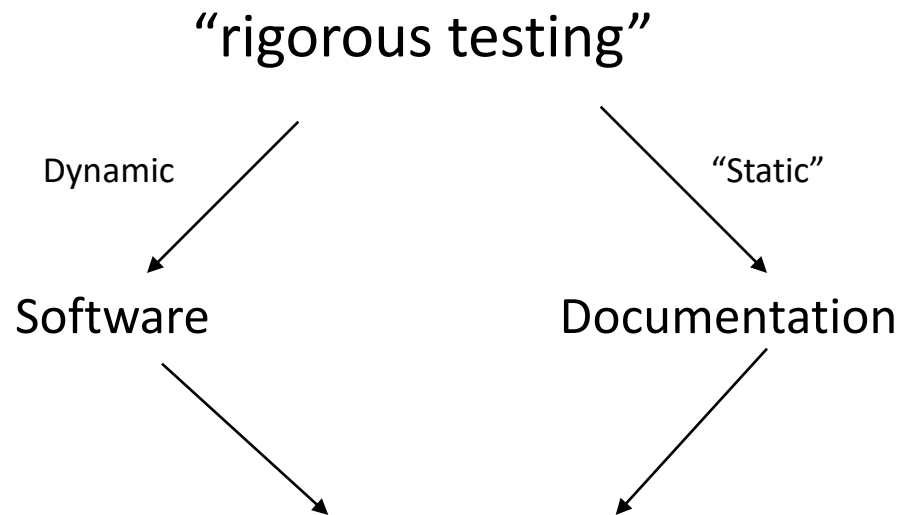
Other factors



Difference between static and Dynamic

- **Static**
 - Checking of work product documentation
- **Dynamic**
 - Execution of the software under test

Testing is important



- Builds confidence
- Reduces risk of operational problems
- Improves software quality

Also Important!

- System meets contractual requirements
- Meets Legal requirements
- Meets industry specific standards

Why is Testing Necessary?

At a shorter run!!

1. Testing finds defects

2. Defects are fixed

3. Testing confirms fixes

4. Software has fewer defects



QUALITY IS IMPROVED!

Why is Testing Necessary?

At a longer run!!

1. Testing finds defects

2. Defect Analysis

- identifies causes

3. Process Improvement

- prevents reoccurrence

4. Future systems have fewer defects



QUALITY IS IMPROVED!

“lessons learned”

Difference between QC & QA

Quality Assurance

The goal is to develop a process so that defects do not arise during production

Plan to avoid the defect in the first place.

It is all about prevention,

It is a proactive process

It is a process based approach

It involves processes managing quality,

Quality audit is an example.

Quality control

It identifies the defects after the product is produced but is not yet released or is still in the production phase.

Try to find defects and correct it.

It is all about the detection.

It is a reactive process.

It is a product based approach.

It is used to verify the quality of the product.

Inspection and testing are examples.

Automation Testing

Content

- What is Automated Testing?
- Automated Testing Advantages
- Limitation of Automated Testing
- Automated testing vs. manual testing
- Automated Test Life-Cycle Methodology (ATLM)
- Automated Testing in CAR Multimedia IAS

What is Automated testing?

- **Automated testing:** The management and performance of test activities, to include the development and execution of test scripts having as objective the verification of test requirements, using an automated test tool.
- The automation of test activities reveals its greatest values in instances where test scripts are repeated or where test scripts subroutines are created and then invoked repeatedly by a number of test scripts.
- Given the continual changes and additions to requirements and software, automated tests serves as an important control mechanism to ensure accuracy and stability of the software through each build.

Automated testing advantages

- Costs and efficiency
 - Detection of the errors that reached production phase (with regression tests)
 - Multiple users simulation
 - Reusable of the old scripts -> creation of the new scripts is reduce
 - automatic execution of performance tests in the beginning of the production -
>less costs for improving the performance
- Time economy
 - quick analysis in case of changing of environment parameters
 - short duration of the testing cycles
 - better estimation for test planning
 - a large number of tests can be executed over night
 - quick generation of testing preconditions
- Quality increase
 - automatic compare of results
 - more consistent results due to repeating tests

Limitation of Automated Testing

- Most of the times an Automated Testing system can't tell if something "looks good" on the screen or when a pictogram or a window is not displayed well
- There are a bunch of problems that can appear when trying to automate the testing process:
 - Unrealistic expectations (e.g. expectation that automated tests will find a lot of errors)
 - Poor testing experience
 - Maintenance of automated tests
- Automated testing will never replace definitely the manual testing
- Tests that should not be automated are:
 - tests that are executed very rare
 - where the system is very unstable
 - tests that can be verified easily manually but hardly automated
 - tests that need physical interaction

Automated testing vs. Manual testing

- Pros of **Automated testing**
 - If a set of tests must be ran repeatedly, automation is a huge win
 - It offers the possibility to run automation against code that frequently change to catch regressions
 - Offers the possibility to add a large test matrix (e.g. different languages on different OS platforms)
 - Automated tests can be run the same time on different machines, whereas manual tests must be run sequentially
 - It offers more time for the test engineer to invoke greater depth and breadth of testing, focus on problem analysis, and verify proper performance of software following modifications and fixes
 - Combined with the opportunity to perform programming tasks, this flexibility promotes test engineer retention and improves his morale
- Cons of **Automated testing**
 - Costs - Writing the test cases and writing or configuring the automate framework that is used costs more initially than running the test manually.
 - Some tests can't be automated – manual tests are needed

Automated testing vs. Manual testing

(2)

- Pros of **Manual testing**

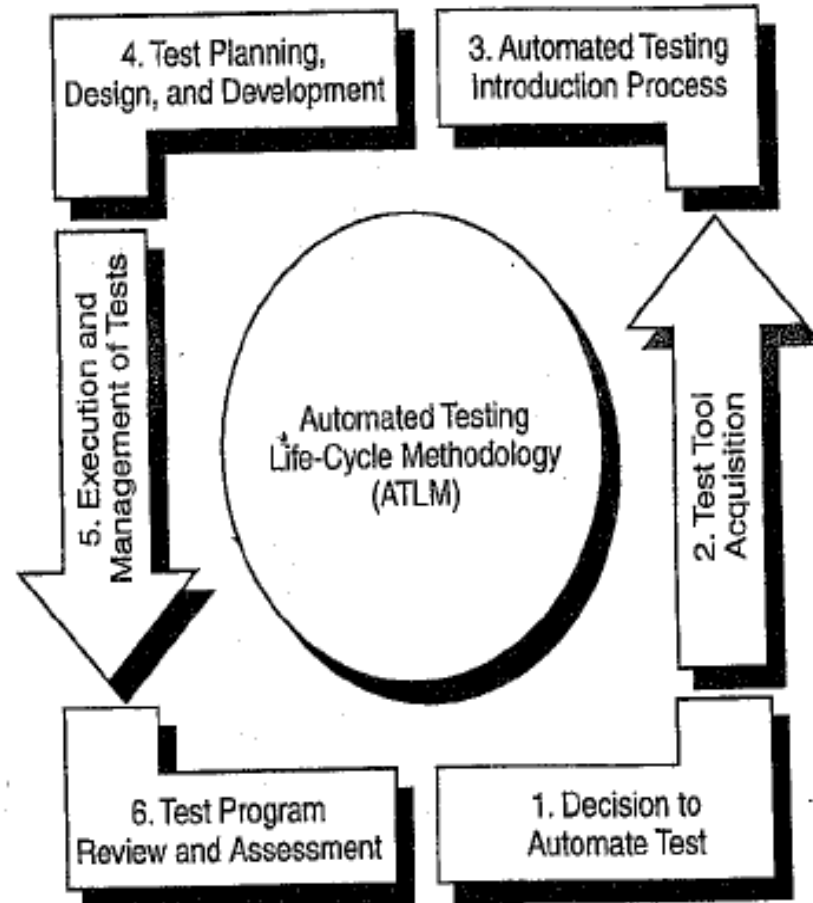
- If the test case runs once or twice most likely is a manual test. Less cost than automating it.
- It allows the tester to perform more ad-hoc (random tests). Experience has proven that more bugs are found via ad-hoc (Experience Based Testing) than via automation testing. And more time the testers spends playing with the feature, the greater are the chances of finding real user problems.

- Cons of **Manual testing**

- Running tests manually can be very time consuming
- The manual tests are requiring more people and hardware
- Each time there is a new build, the tester must rerun all required tests - which after a while would become very boring and tiresome.

Automated Test Life-Cycle Methodology (ATLM)

1. Decision to Automate Test
2. Test Tool Acquisition
3. Automated Testing Introduction Process
4. Test Planning, Design and Development
5. Execution and Management of Tests
6. Test Program Review and Assessment



Decision to Automate Test: Overcoming False Expectations

EXPECTATION

- Automatic test plan generation
- Test tool fits all
- Imminent test effort reduction
- Tool ease of use
- Universal application of test automation
- 100% test coverage

REALITY

- No tool can create automatically a comprehensive test plan
- No single tool can support all operating systems environments and programming languages
- Initial use of an automated test tool can actually increase the test effort
- Using an automated tool requires new skills, additional training is required
- Not all the tests required for a project can be automated (e.g. some test are physically impossible; time or cost limitation)
- It is impossible to perform an exhaustive testing of all the possible inputs (simple or combination) to a system

Decision to Automate Test: Benefits of Automated Testing

- | | |
|---|---|
| ▶ Production of a reliable system | ▶ Improved requirements definition |
| | ▶ Improved performance/load/stress testing |
| | ▶ Improved partnership with development team |
| | ▶ Improved system development life cycle |
| ▶ Improvement of the quality of the test effort | ▶ Improved build verification testing (smoke testing) |
| | ▶ Improved regression testing |
| | ▶ Improved multiplatform/software compatibility testing |
| | ▶ Improved execution of the repetitive tests |
| | ▶ Improved focus on advanced test issues |
| | ▶ Execution of tests that manual testing can't accomplish |
| | ▶ Ability to reproduce software defects |
| | ▶ After-hours testing |
| ▶ Reduction of test effort and minimization of schedule | |

Test Tool Acquisition

- Identify which of the various tool types suit the organization system environment, considering:
 - the group/department that will use the tool;
 - the budget allocated for the tool acquisition;
 - the most/least important functions of the tool etc.
- Choose the tool type according to the stage of the software testing life cycle
- Evaluate different tools from the selected tool category
- Hands-on tool estimation – request product demonstration (evaluation copy)
- Following the conclusion of the evaluation process, an evaluation report should be prepared

Business analysis phase	▶ business modeling tools ▶ configuration management tools ▶ defect tracking tools
Requirements definition phase	▶ requirements management tools ▶ requirements verifiers tools
Analysis and design phase	▶ database design tools ▶ application design tools
Programming phase	▶ syntax checkers/debuggers ▶ memory leak and run-time error detection tools ▶ source code or unit testing tools ▶ static and dynamic analyzers
Testing phase	▶ test management tools ▶ network testing tools ▶ GUI testing tools (capture/playback) ▶ non-GUI test drivers ▶ load/performance testing tools ▶ environment testing tools

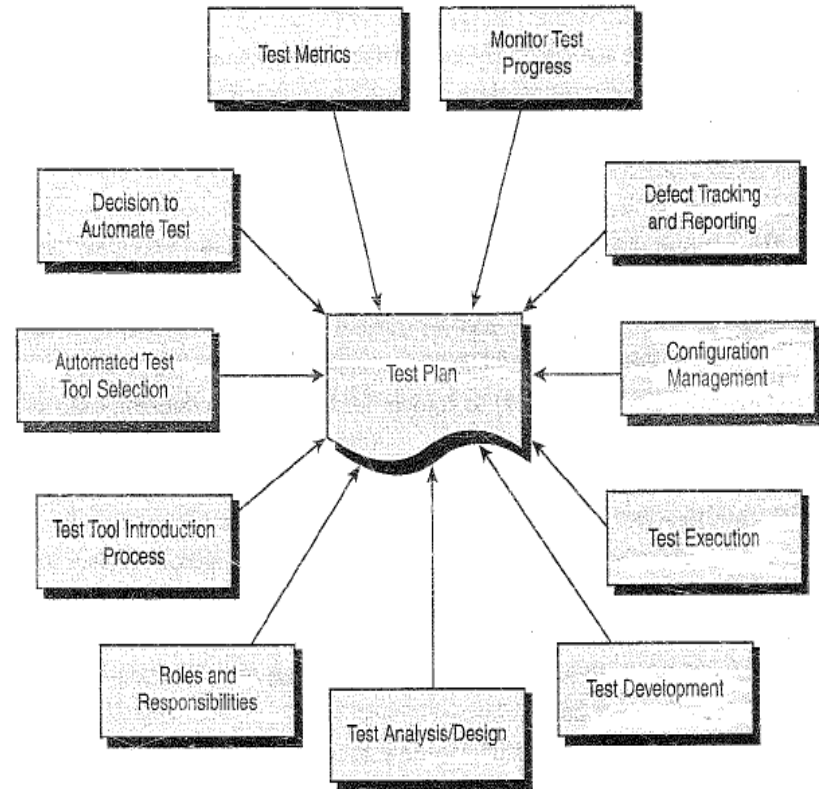
Automated Testing Introduction Process

– Test Process Analysis

Process review	▶ Test process characteristics (goals, strategies, methodologies) have been defined and they are compatible with automated testing ▶ Schedule and budget allows process implementation ▶ The test team is involved from the beginning of SDLC
Test goals	▶ Increase the probability that application under test will behave correctly under all circumstances ▶ Increase the probability that application meets all the defined requirements ▶ Execute a complete test of the application within a short time frame
Test objectives	▶ Ensure that the system complies with defined client and server response times ▶ Ensure that the most critical end-user paths through the system perform correctly ▶ Incorporate the use of automated test tools whenever feasible ▶ Perform test activities that support both defect prevention and defect detection ▶ Incorporate the use of automated test design and development standards to create reusable and maintainable scripts
Test strategies	▶ Defect prevention (early test involvement, use of process standards, inspection and walkthroughs) ▶ Defect detection (use of automated test tools, unit/integration/system/acceptance test phase)

Test Planning

- The test planning element of the ATLM incorporates the review of all activities required in the test program
- It ensures that testing processes, methodologies, techniques, people, tools, schedule and equipment are organized and applied in an efficient way
- Key elements: planning associated with project milestone events, test program activities and test program-related documentation.
- The following must be accomplished:
 - the technical approach for these elements is developed;
 - personnel are assigned
 - performance timelines are specified in the test program schedule.
- Test planning is not a single event, but rather a process. It is the document that guides test execution through to a conclusion, and it needs to be updated frequently to reflect any changes.



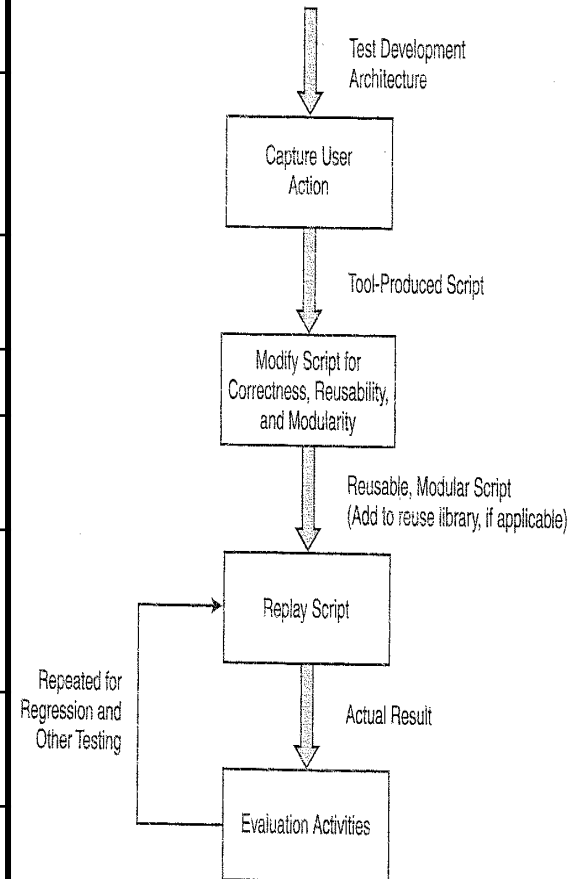
Test Design

- Test program model definition – identifies the test techniques that apply to the test program
- Test architecture definition – involves the selection of a test architecture model and population of model attributes
- Test procedure definition – groups logically the test procedures
- Automated/manual test mapping (decision what to automate)

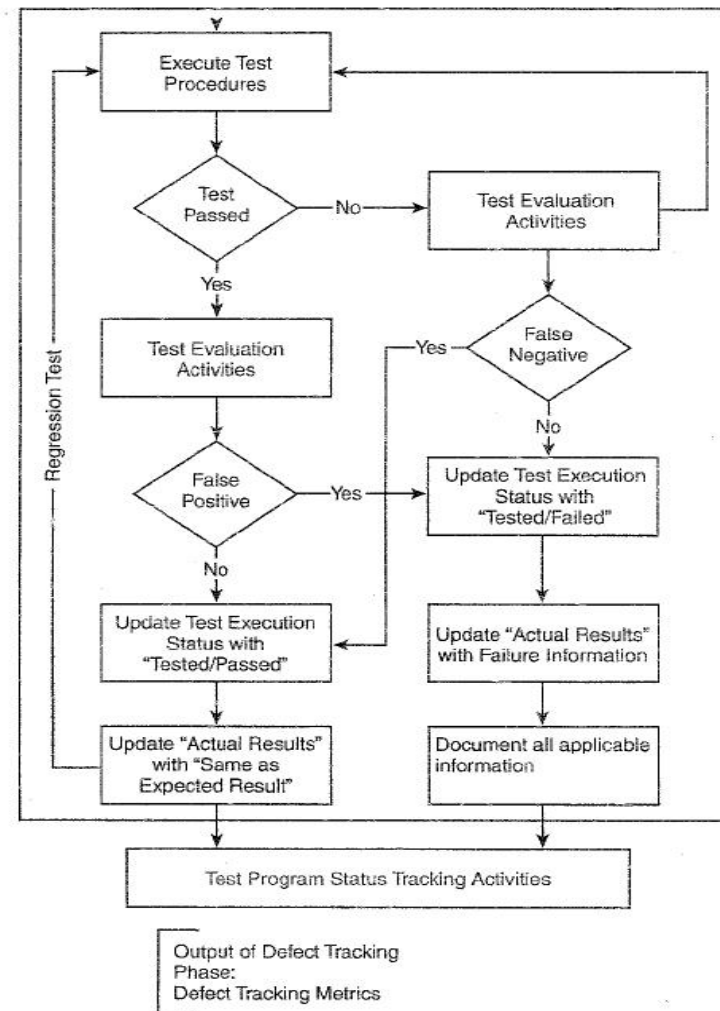
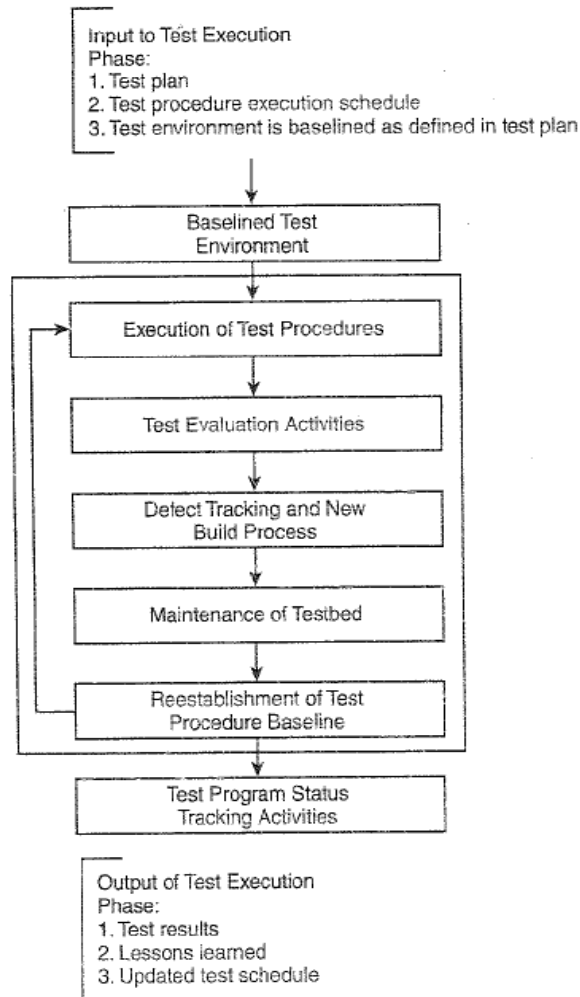
Black-Box Test Techniques	Automated Test Tools
▶ Random Testing	▶ GUI Test Tools
▶ Regression Testing	▶ GUI/Server Test Tools
▶ Stress/Performance Testing	▶ Load Test Tools
▶ Security Testing	▶ Security Test Tools
▶ Data Integrity Testing	▶ Data Analysis Tools
▶ Configuration Testing	▶ Multiplatform Test Tools
▶ Functional Testing	▶ Load/GUI/Server Test Tools
▶ User Acceptance Testing	▶ GUI Test Tools
▶ Usability Testing	▶ Usability Measurement Tools
▶ Alpha/Beta Testing	▶ Load/GUI/Server Test Tools
▶ Boundary Value Analysis	▶ Develop program code to perform tests
▶ Backup and	▶ Load/GUI/Server Test

Test Development

<u>Development Guideline Topics</u>	<u>Description</u>
Design-to-Development Transition	Specify how design and setup activities will be translated into test development action
Reusable Test Procedures	Test procedures need to be reusable for highest test program return on investment
Data	Avoid hard-coding data values into scripts
Capture/Playback	Outlines on how to apply the use of capture/playback recording
Maintainable Test Procedures	A test procedure whose defects are easy to remove and can easily be adapted to meet new requirements
Test Script Documentation	Test script documentation is important for test procedure maintainability
Naming Standards	Defines the standard naming convention for test procedures
Modularity	Guidelines for creating modular test scripts
Global Files	Globally declared functions are available to any procedure
Constants	Use of constants in order to support maintainable



Test Execution



TTCN-3

- **TTCN-3** is the **T**esting and **T**est **C**ontrol **N**otation version **3**
- **Internationally standardized** testing language for formally defining test scenarios – design purely for testing
 - developed from 1999 – 2002 at the *European Telecommunications Standards Institute* (ETSI)

▶ A programming language that has been used for more than 20 years in standardization as well as industry

▶ Area of Testing

- Conformance and Functionality Testing
- Pre-integration Testing
- Integration and Interoperability Testing

```
testcase Hello_Bob () {  
    p.send("How do you do?");  
  
    alt {  
        [] p.receive("Fine!") {  
            setverdict(pass);  
        }  
        [else] {  
            setverdict(inconc); // Bob asleep!  
        }  
    }  
}
```

Advantages of using TTCN-3

- Systematic product testing
- Large percentage of testing coverage
- Parallel testing and develop processes
- Automated testing so it can be run without user intervention
- Complex and multiple scenarios implementation
- Clear and exact bug reports
- Easy management of errors and bug fixing
- Reusable tests and consistent version validation
- Automated regression tests on all levels
- Control and exact status of product quality in all development stages

Other Automated Test Tools

Mercury Interactive - www.mercuryinteractive.com	Application performance management	▶ <u>Topaz</u> – solution for performance management from the end-user perspective: real time alerts receiver, user problems correlation with system data, service levels management
	Test management	▶ <u>TestDirector</u> – global solution for test process management, containing four modules: Requirements, Test Plan, Test Lab, Defects Manager
	Functional testing	▶ <u>Astra FastTrack</u> – defect management tool for Web applications ▶ <u>Astra QuickTest</u> – tool that allows both the beginner and experienced testers to test the dynamic Web applications ▶ <u>WinRunner</u> – functional testing solution for GUI applications
	Load testing	▶ <u>QuickTest professional</u> - tool for automation process of functional and regression testing of the dynamic Web applications ▶ <u>XRunner</u> – automation functional testing of the X-Window applications ▶ <u>LoadRunner</u> – load and performance testing management ▶ <u>Astra LoadTest</u> – quick solution for performance and scalability testing of the Web applications ▶ <u>LoadRunner TestCenter</u> – complete solution that allows multiple and concurrent testing management, from different locations. All the resources are centralized and accessible from a single control center

Other Automated Test Tools (2)

Segue - www.segue.com	Application performance management	▶ SilkVision – solution for application behavior monitoring ▶ SilkTest – tool for application scalability and stress testing ▶ SilkPerformer – simulates the application performance and stress testing ▶ SilkPlan – process management package
Rational – www.rational.com	Application performance management Test management	▶ Rational Suite TestStudio ▶ Rational Test RealTime ▶ Rational TestManager
Compuware Corporation www.compuware.com	Application performance management	▶ Abend-AID ▶ File-AID ▶ PointForward ▶ QACenter ▶ XPEDITER
	Production Readiness Quality Assurance Development and Integration	▶ Abend-AID ▶ NuMega ▶ QA Center ▶ File-AID ▶ Uniface ▶ Vantage

```
graph LR; A[Source Code] --> B[Compiler]; B --> C[Object Code/Machine Code/Byte Code]; C --> D[Interpreter];
```

SOURCE CODE

C
O
M
P
I
L
E
R

OBJECT CODE/MACHINE
CODE/BYTE CODE

I
N
T
E
R
P
R
E
T
E
R